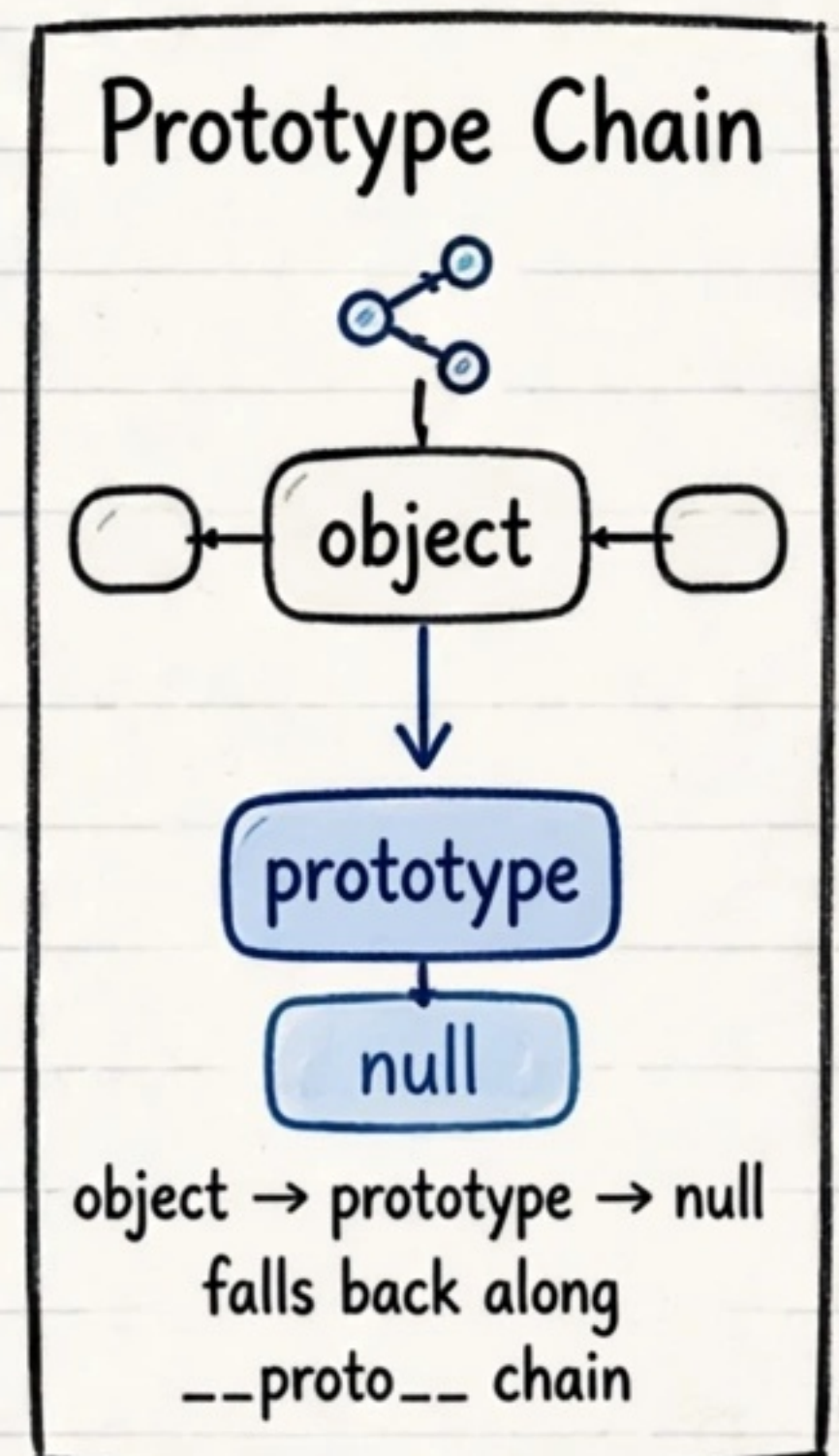
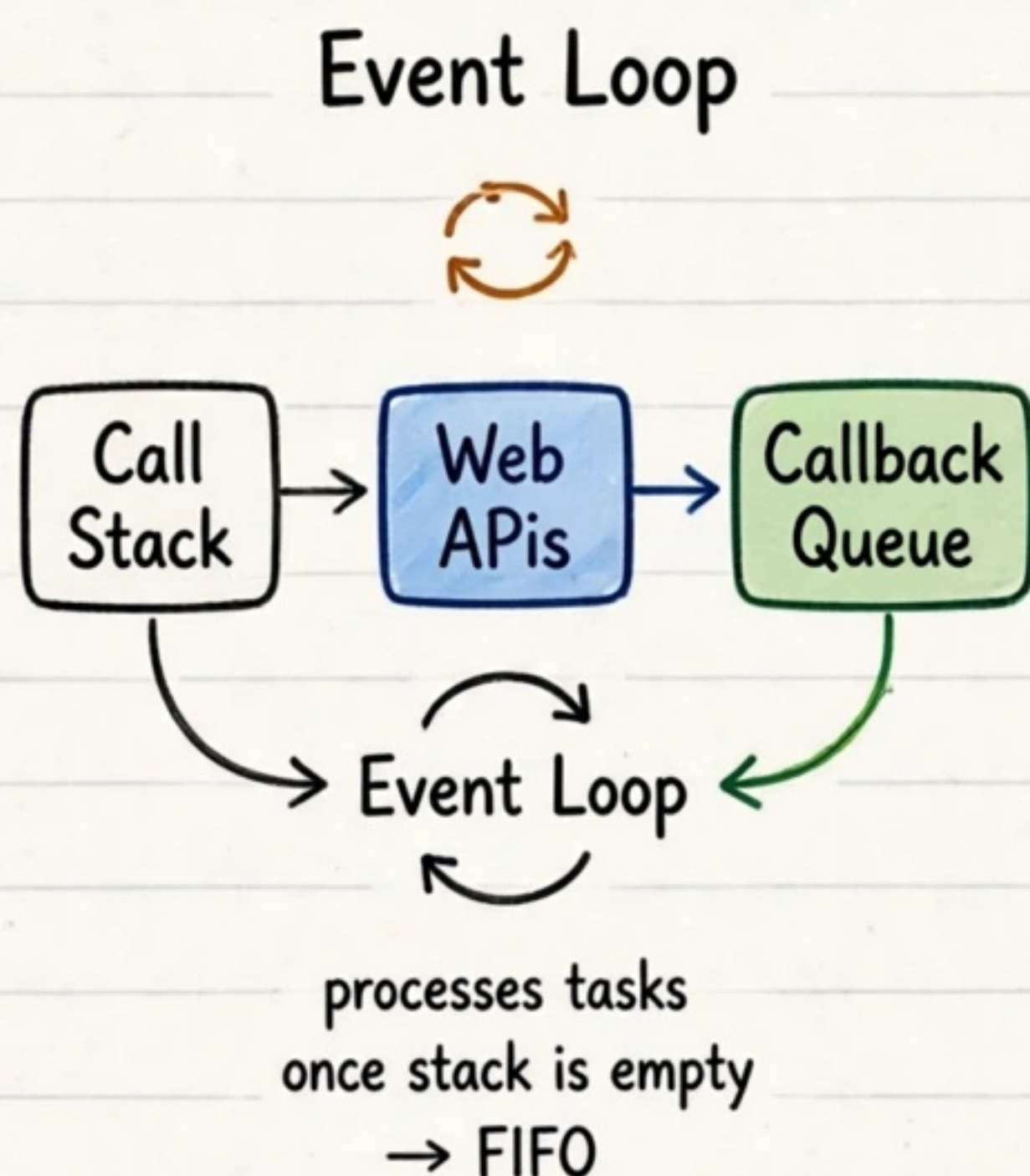
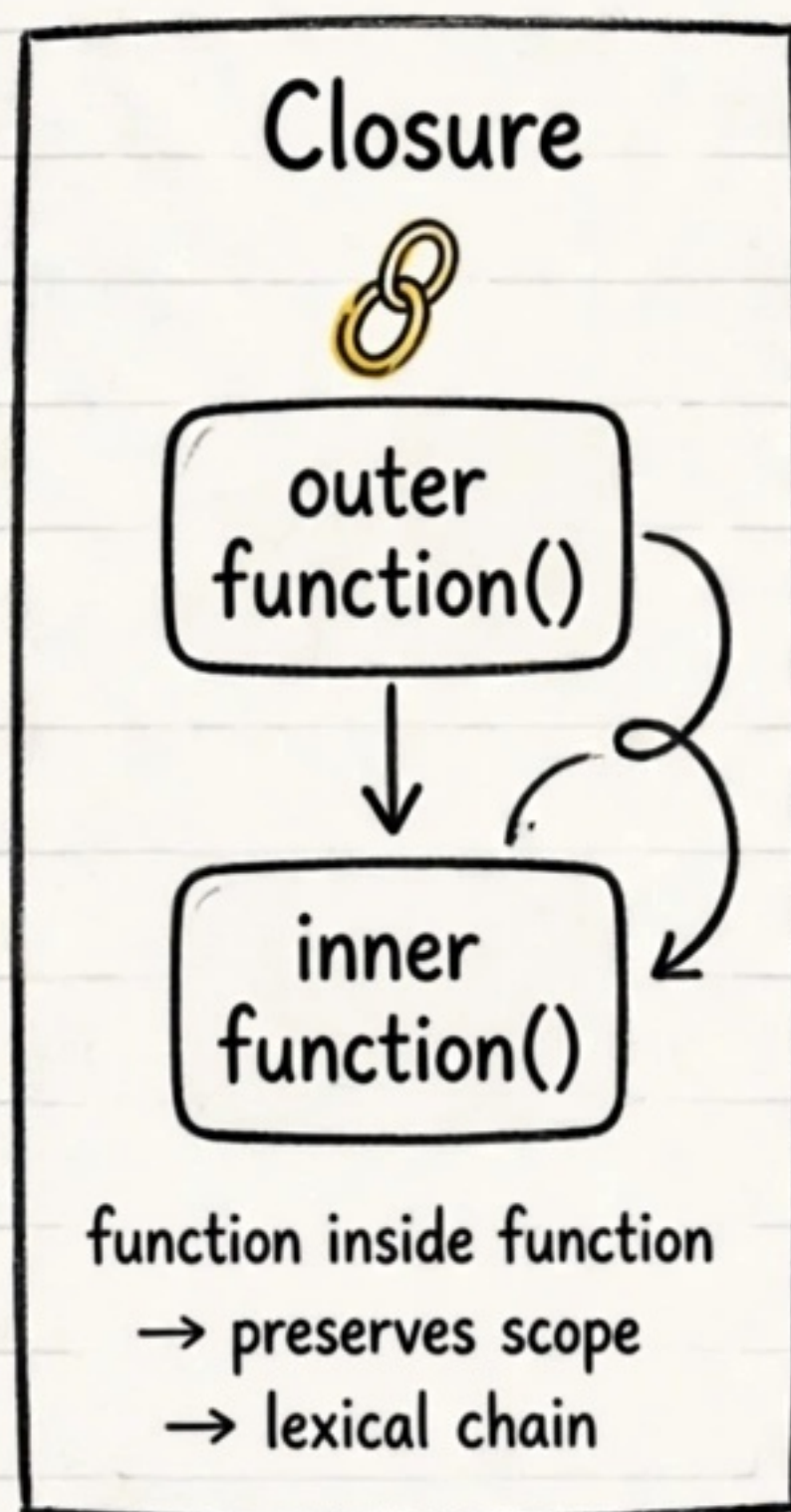




Top 15 Must-Know JavaScript Concepts Visual Handbook

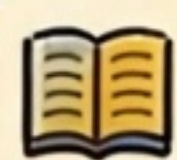
✦ Master these concepts to crack Frontend Interviews

Visual Learning Diagram



★ Why Learn JS Concepts?

- Build strong fundamentals
- Crack frontend interviews
- Understand how JS works internally
- Avoid common bugs
- Used by top companies



In this handbook you'll learn:

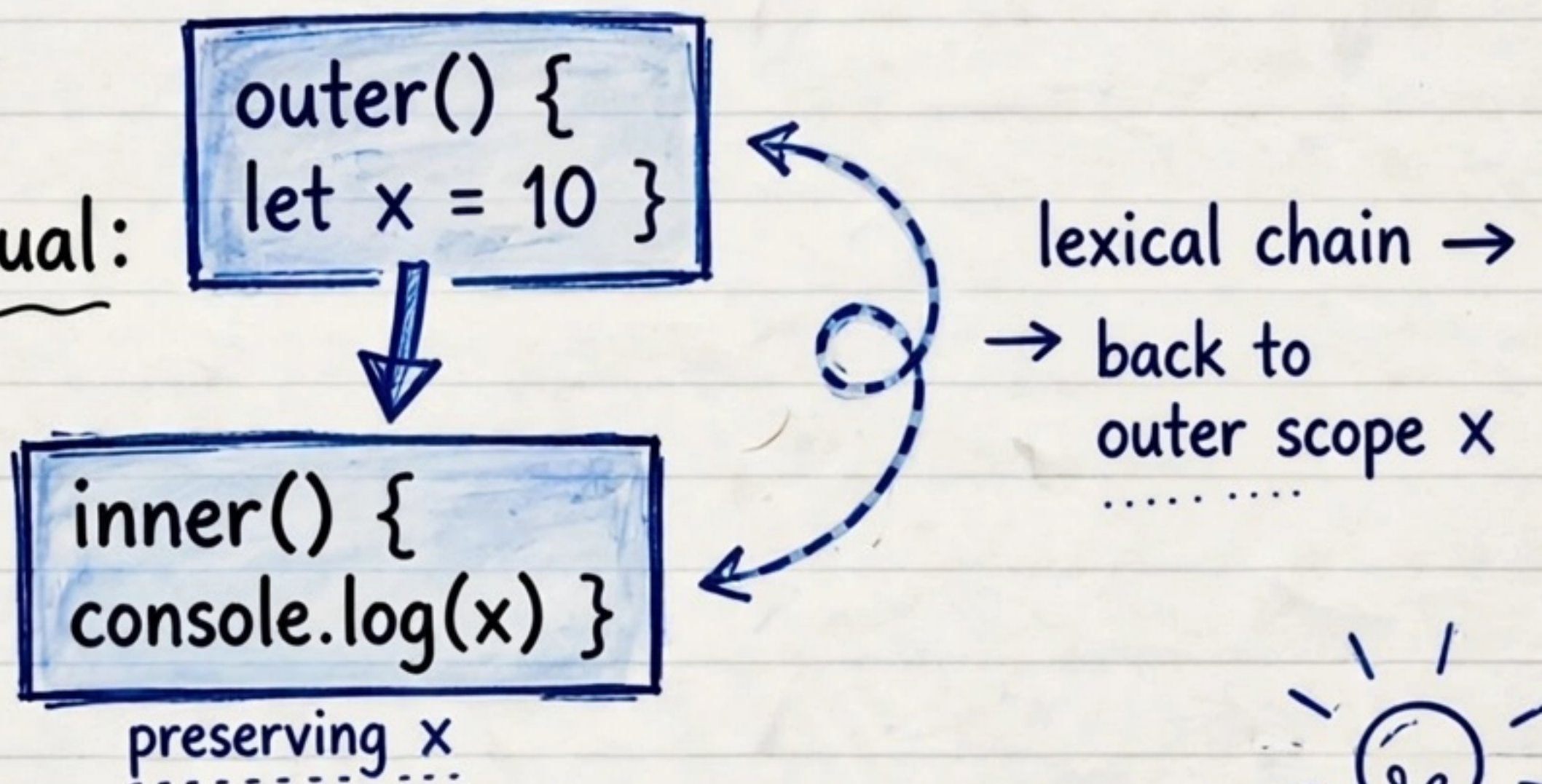
- | | | |
|---------------------------|------------------------|-----------------------|
| 1. Closures | 6. This keyword | 11. ES6 Features |
| 2. Hoisting | 7. Scope Chain | 12. Event Delegation |
| 3. Event Loop | 8. Currying | 13. Shadowing |
| 4. Promises & Async/Await | 9. Debounce & Throttle | 14. Memoization |
| 5. Prototypes | 10. Call/Apply/Bind | 15. Final Cheat Sheet |

Swipe to Page →

Closures

★ Overview: Closure is inner function remembering outer scope even after outer finished — preserves scope via lexical chain.

Step-by-step visual:



① Step 1:

`outer()` called,
`x = 10` created in
outer scope

outer scope:
`x = 10`

② Step 2:

`inner()`
returned,
outer finished
but `x` preserved
in closure → 

③ Step 3:

`inner()`
called later
→ still accesses
`x = 10 =>`
`=> FOUND`
✓ closure ✓

★ Why use Closures?

- Data privacy / encapsulation
- Currying & partial application
- Memoization & once
- Event handlers & callbacks

HOW TO RECOGNIZE THE PATTERN:

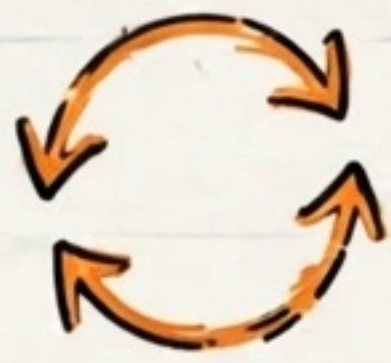
- ☒ Function returned from another function
- ☒ uses outer variable after outer finished
- ☒ Data privacy
- ☒ Currying
- ☒ Keywords: `inner` • `outer` • lexical scope



COMPLEXITY & DATA:

- Time $O(1)$ access • Space $O(n)$ closure retains ref
- Common pitfalls: loop var bug, memory leak
- Interview tips: private counter, module pattern
- Examples: Counter, Debounce, Memoize

→ Swipe to Page 03/15



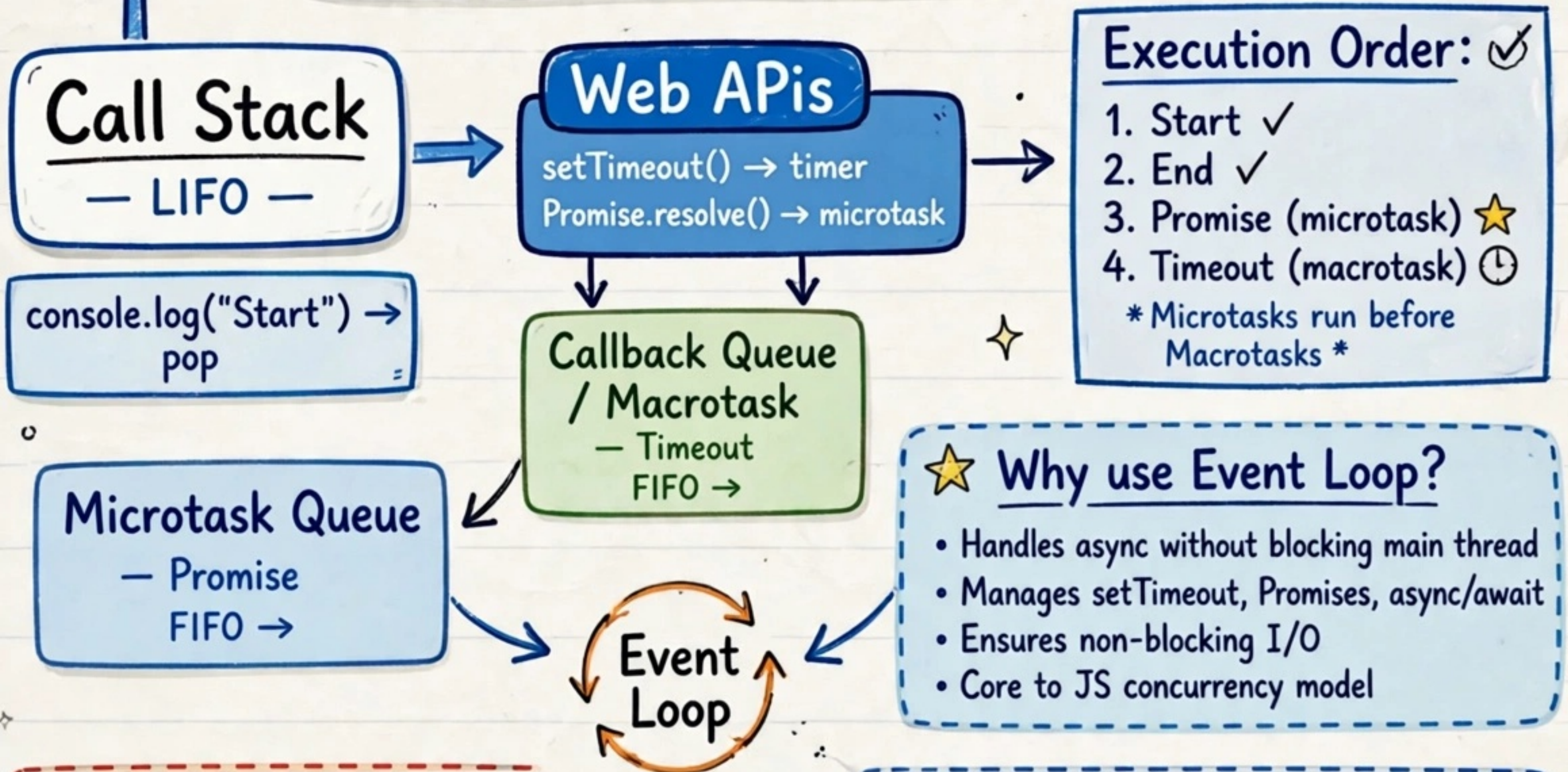
Event Loop

★ Overview: JS is single-threaded, Event Loop processes tasks from queues once Call Stack empty — handles async via Web APIs.

Step-by-Step Visual Diagram:

CODE:

```
console.log("Start");
setTimeout() => console.log("Timeout"), 0);
Promise.resolve().then(); => console.log("Promise"));
console.log("End");
```



✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ `setTimeout` / `setInterval` present
- ✓ Promise / `fetch` / `async` code
- ✓ Interview asks order of execution
- ✓ Need to explain sync vs async
- ✓ Keywords: stack • queue • microtask • macrotask • event loop • Web APIs

processes queue → moves to Call Stack when empty



COMPLEXITY & DATA:

- Microtasks (Promise) run BEFORE Macrotasks (`setTimeout`)
- Call Stack LIFO, Queues FIFO
- Common pitfalls: blocking stack, starving microtasks
- Interview tip: `Promise.then` > `setTimeout` even with 0ms
- Example output: Start, End, Promise, Timeout
- Time $O(n)$ queue processing

→ Swipe to Page 04/15



Hoisting

★ Overview: Hoisting moves declarations to top during creation phase - var = undefined, let/const in Temporal Dead Zone.

Step-by-step visual: Execution Context - Creation vs Execution

CODE EXAMPLE:

```
console.log(a); // undefined
console.log(b); // ReferenceError
var a = 5;
let b = 10;
sayHi(); // works!
function sayHi()= console.log("Hi"); }
```

Phase 1: CREATION PHASE [Memory Allocation] ↓

var var a = undefined
→ value is undefined

let b = <unintilized> TDZ
⚠ TDZ - Temporal Dead Zone

✓ function sayHi = fully hoisted
→ available immediately

↓ Phase 2: EXECUTION PHASE

→ Actual code runs line by line in order

★ WHY UNDERSTAND Hoisting?

- Avoid undefined bugs & ReferenceError
- Understand JS execution context
- Explain var vs let/const in interviews
- Debug tricky scope issues

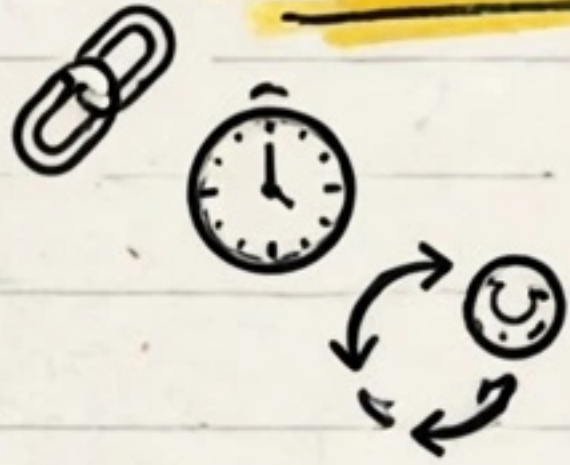
✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ var before declaration → undefined
- ✓ let/const before → ReferenceError (TDZ)
- ✓ function before declaration → works
- ✓ Function expr (let/const) → not hoisted
- ✓ Keywords: hoisting, TDZ, undefined, creation phase

💡 COMPLEXITY & DATA:

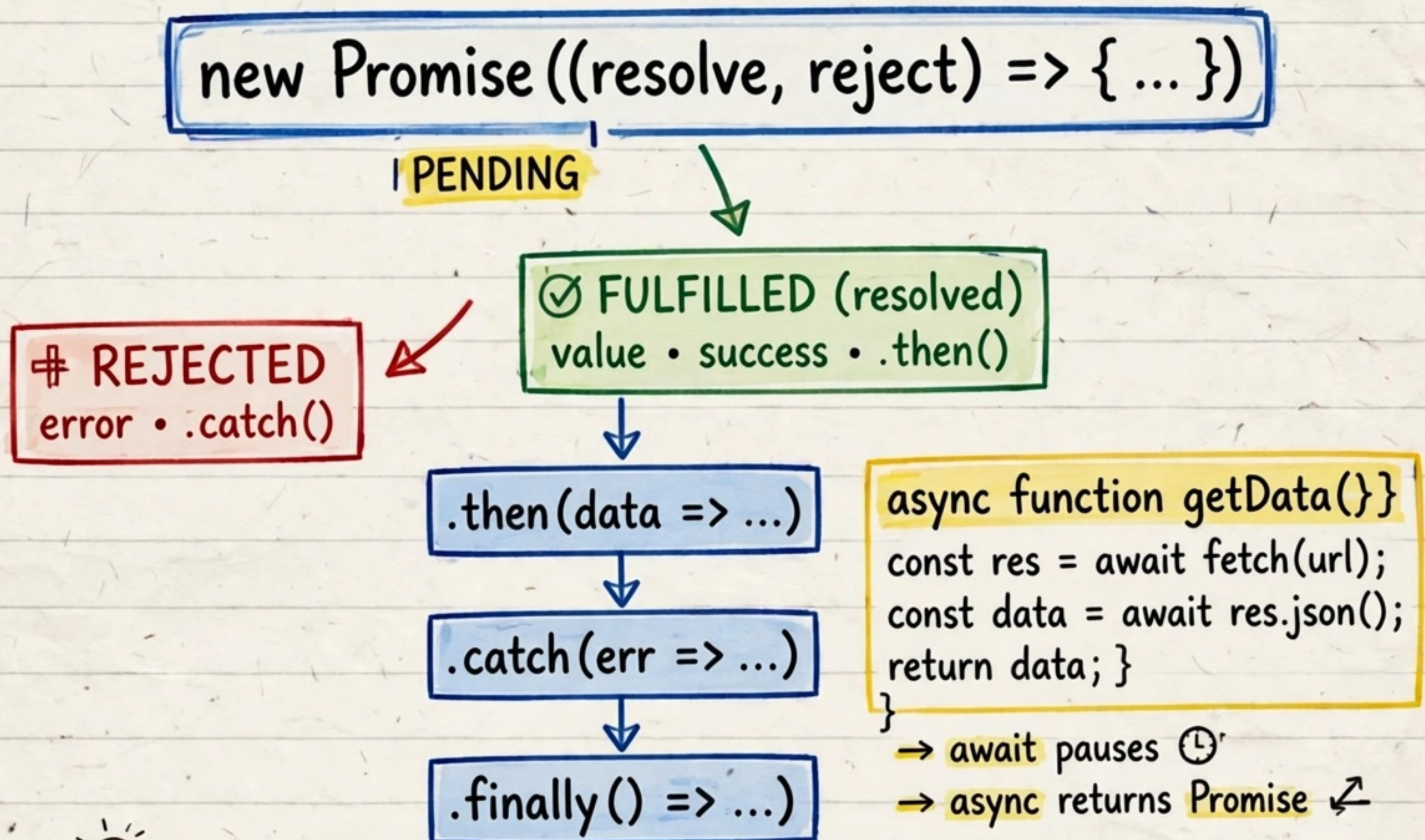
- Time $O(1)$ • Space $O(1)$ - compile time behavior
- Common Pitfall: using var causes undefined bugs
- Interview Tip: Always use let/const, explain TDZ
- Examples: var hoist, let TDZ error, function hoisting
- Execution Context = Memory (Creation) + Code (Execution)

Promises & Async/Await



★ **Overview:** Promise represents future value - 3 states: pending, fulfilled, rejected, async/await is syntactic sugar making async code look sync. `.then/.catch/.finally` for handling.

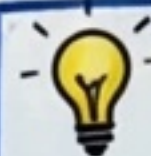
Step-by-step visual:



★ Why use Promises & Async/Await?

✔ HOW TO RECOGNIZE THE PATTERN:

- ✔ Callback hell to avoid
- ✔ `.then` chain / promise chain
- ✔ `fetch` API / API calls
- ✔ async function returns promise
- ✔ Keywords: pending, resolved, rejected, await, then



COMPLEXITY & DATA:

- Time: depends on async op, non-blocking
- Space $O(1)$ for chain
- Common pitfalls: forgetting `await`, unhandled rejection
- Interview tip: `Promise.all` vs `Promise.race`
- Examples: `fetch` data, `setTimeout` promisified, chaining APIs
- Tip: always use `try/catch` with `await`

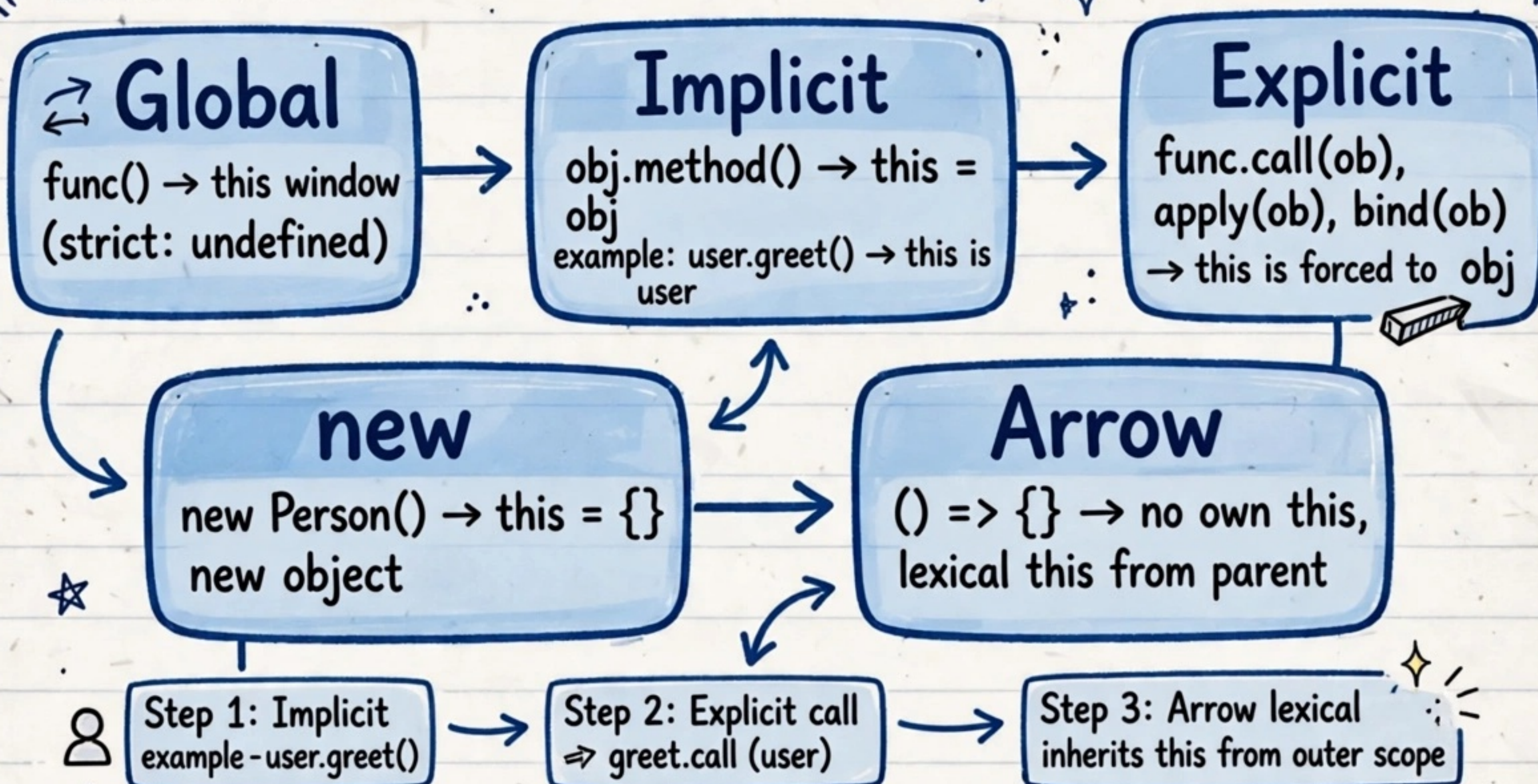
→ Swipe to Page 06/15



This Keyword

☆ Overview: this refers to execution context - depends how function called: global, implicit obj.method(), explicit call/apply/bind, new keyword, arrow lexical (no own this).

Step-by-step visual:



☆ Why use This?

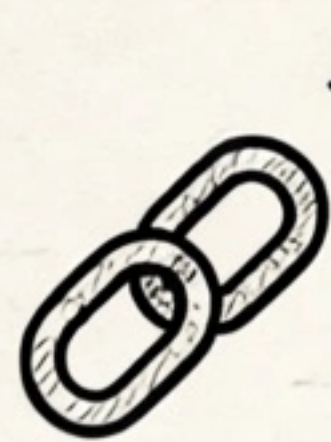
☑ HOW TO RECOGNIZE THE PATTERN:

- ✓ obj.method() → this is obj
- ✓ standalone func() → window / undefined
- ✓ arrow fn has no own this - uses parent
- ✓ new Fn() → this is new instance
- ✓ Keywords: bind / call / apply / new



COMPLEXITY & DATA:

- Time $O(1)$ · Space $O(1)$ binding
- Pitfalls: lost this in callbacks, arrow in methods
- Interview tips: explain 4 bindings order (new > explicit > implicit > default)
- Examples: event handlers, class methods, constructor
- Arrow lexical solves callback this bug

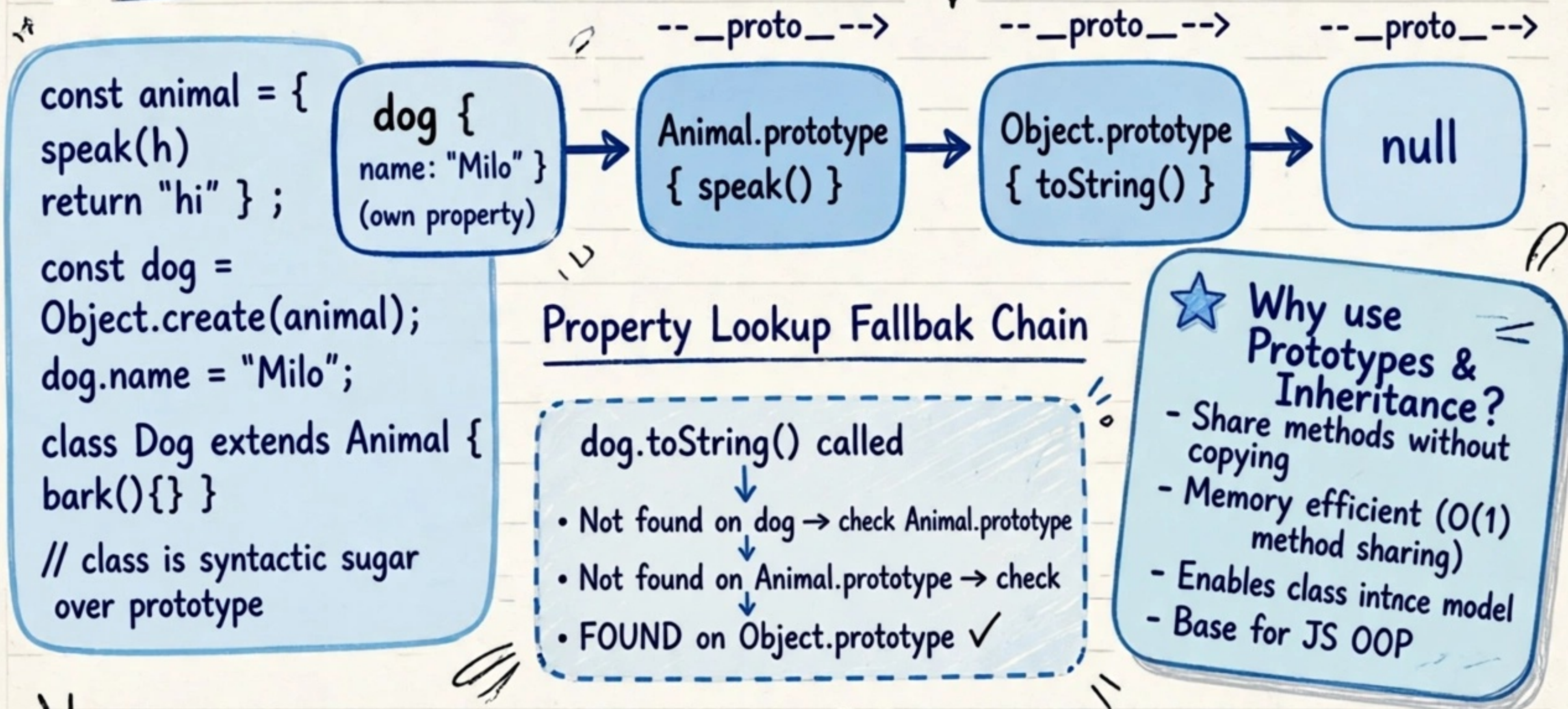


Prototypes & Inheritance

☆ Page 07/15

☆ Overview: Every JS object has hidden `__proto__` linking to prototype - forms chain ending at null for property fallback.

Step-by-step visual:



✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ Object.create (proto)
- ✓ class extends
- ✓ inespuep checks prototype chain
- ✓ Methods shared via prototype
- ✓ Keywords: `__proto__`, prototype, chain, inheritance



COMPLEXITY & DATA:

- Lookup Time: $O(\text{chain length})$
- Space: $O(1)$ shared
- Pitfall: Mutating prototype affects all instances
- Don't use `__proto__` directly; use `Object.getPrototypeOf()`
- Interview: Explain `dog.__proto__ === Animal.prototype`
- Examples: `Array.map()`, custom inheritance, polyfills

→ Swipe to Page 08/15

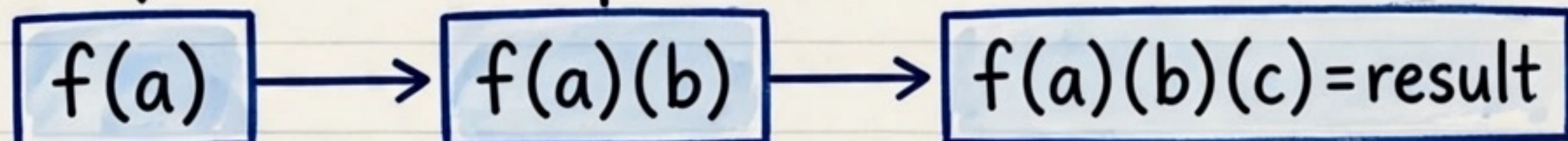
Currying + Debounce & Throttle

- ★ Overview: Currying transforms multi-arg fn to chain of unary fns
 - Debounce delays, Throttle limits rate for performance.

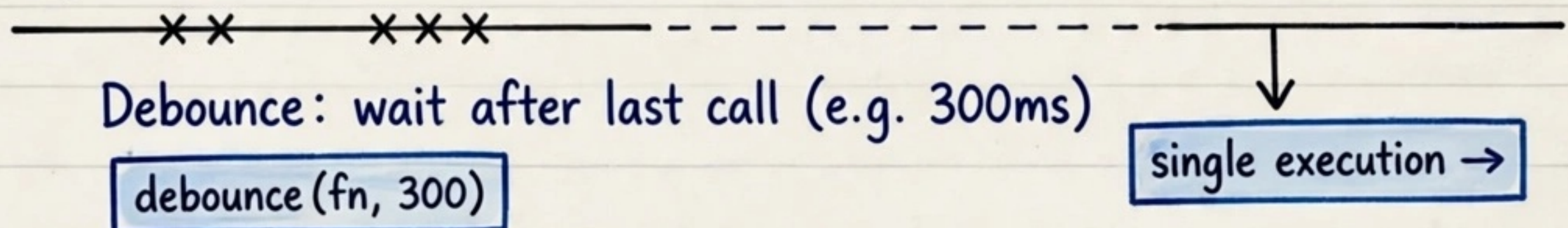
Step-by-step visual:

1) CURRYING:

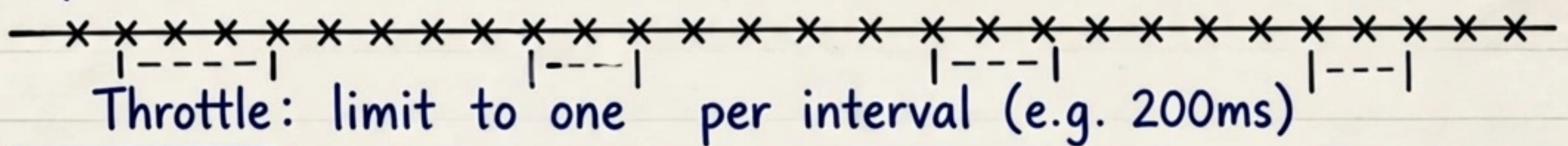
```
function curry(f){return a=>b=>c=>f(a,b,c)}
```



2) DEBOUNCE TIMELINE:



3) THROTTLE TIMELINE:



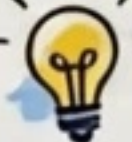
`throttle(fn, 200)`

- ★ Why use Currying / Debounce & Throttle?
 - Partial application & reuse



HOW TO RECOGNIZE THE PATTERN:

- ✓ Search input debounce
- ✓ Scroll / resize throttle
- ✓ Partial application needed
- ✓ Interview asks performance optimization
- ✓ Keywords: delay • limit • rate • wait • curry



COMPLEXITY & DATA:

- Currying: Time $O(n \text{ args})$
- Space $O(n)$ closures
- Debounce/Throttle: $O(1)$
- Pitfall: losing this/context → use arrow/bind
- Leading vs trailing option
- Example: Lodash `_.debounce`, `_.throttle`
- Tip: `cancel()` method

→ Swipe to Page 09/15

★ Overview: Explicit binding - call and apply invoke immediately with this, bind returns new function with fixed this for later.

Step-by-step visual:

Call Apply Bind

```
const obj = { name: "JS" }
```

```
function greet(a,b){  
  return this.name + a + b }  
↓ thisArg → obj
```

CALL

```
greet.call(obj,  
  "Hi", !!")  
↓  
→ Invoke immediately,  
args comma separated
```

APPLY

```
greet.apply(obj,  
  ["Hi", !!"])  
↓  
→ Invoke immediately,  
args as array
```

BIND

```
const newFn =  
  greet.bind(obj, "Hi")  
↓  
→ Returns new function,  
call later  
newFn("!!")
```

call → a, b - apply → [a, b] - bind → returns fn

★ Why use Call Apply Bind?

- Borrow methods between objects
- Control this explicitly
- Partial application / currying with bind
- Fix this loss in callbacks

✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ Borrow method from other object
- ✓ Fix this loss in callbacks / events
- ✓ Partial application via bind(a,b)
- ✓ Need explicit this setting
- ✓ Keywords: explicit binding, borrowing, thisArg, loan method



COMPLEXITY & DATA:

- Time $O(1)$ for binding, call depends on fn
- Space $O(1)$, bind creates new fn
- Common pitfalls: bind doesn't invoke, arrow fns ignore this
- Interview tips: Explain call vs apply vs bind + polyfill
- Examples: Function borrowing, event handlers, currying
- Real use: obj.method.call (otherObj)

→ Swipe to Page 10/15



Scope Chain & Lexical Environment

★ Overview: Scope determines variable access - chain looks outward: local → outer → global, block scope with let/const.

Global Scope

let globalVar = "I am Global";

Function Scope: outer()

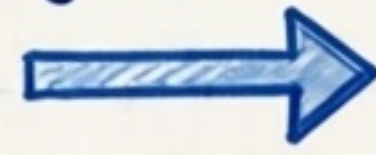
let outerVar = "I am Outer";

Block Scope { inner }

let innerVar = "I am Inner";
console.log(globalVar)
// found via chain;
console.log(outerVar)
// found via chain;



inner →
outer →
global



Chain looks outward



SHADOWING EXAMPLE:

```
let x = 10;
{ let x = 20;
  console.log(x) //
  20 shadows outer }
console.log(x) // 10
```

★ Why use Lexical Scope?

- Controls variable access & privacy
- Enables closures
- Prevents global pollution
- Block scope with let/const = safer code

✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ let & const are block scoped {}
- ✓ var is function scoped
- ✓ Inner can access outer, not vice versa
- ✓ Shadowing: inner var hides outer
- ✓ Keywords: lexical, scope chain, block, closure, shadowing



COMPLEXITY & DATA:

- Lookup: $O(n)$ chain depth (usually $O(1)$)
- Space: $O(n)$ for environment records
- Pitfall: var hoisting, accidental global
- Interview Tip: Explain TDZ for let/const
- Example Qs: scope chain output, shadowing, TDZ

✨ ES6+ Features

- ★ Overview: ES6 modern syntax – makes JS concise: destructuring, spread, arrow, template literals, ?. ??, modules import/export.

Step-by-step visual:

- Destructuring**
`const {name, age} = user`
↓
`user { name: 'Alex', age: 25 }` → `{ name: 25 }` → `name → 'Alex', age → 25`
- Spread/Rest '... operator'**
`const newArr = [...arr, 4]`
↓
`function sum(...args) { ... }`
- Arrow Functions**
`const add = (a,b) => a + b`
↓
• lexical this - no own this
- Template literal & Nullish/Optional**
↓
``Hello ${name}``
`user?.address?.city` → returns undefined
`value ?? "default"` → nullish coalesce if null/undefined
→ fallback if null/undefined
- Modules**
↓
`import { x } from "../mod"` → `export const x = 42`
share code between files

☆ Why use ES6+ Features?

- Makes code concise → cleaner & readable
- Avoids mutation & errors → safer data handling
- Lexical this with arrow → predictable scope
- Modern interview must-know → standard in 2024+
- Works everywhere now → supported in all modern JS

✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ ... spread / rest operator
- ✓ Destructuring {a,b} [a,b]
- ✓ Arrow => lexical this
- ✓ Template {{f}}`
- ✓ ?. ?? operators
- ✓ Keywords: import export let const

COMPLEXITY & DATA:

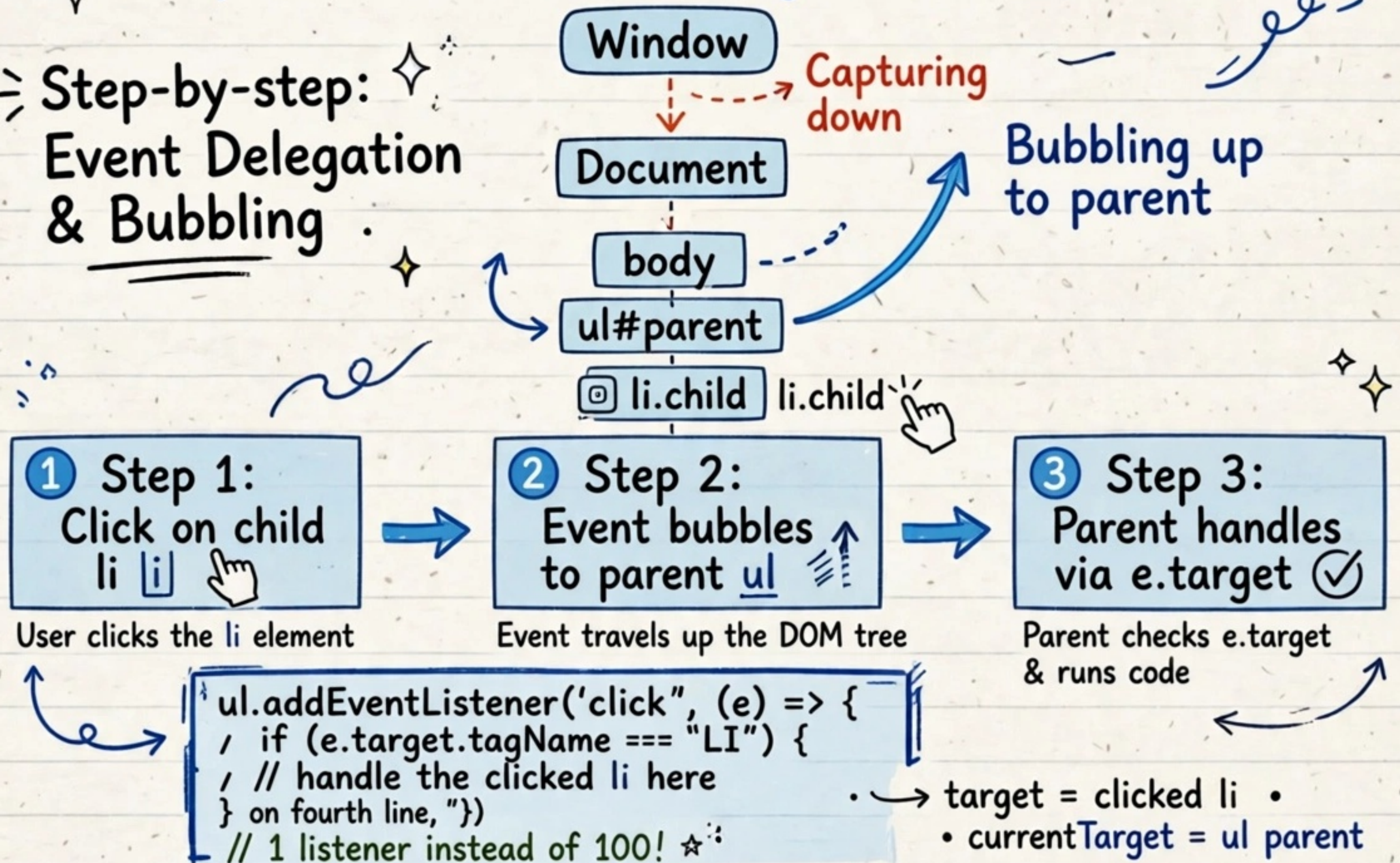
- Time: $O(1)$ — syntax sugar, no runtime cost
- Space: $O(1)$ — no extra memory
- Common pitfalls: ... mutates shallow copy only
- ?? vs || → ?? only for null/undefined
- ?. short-circuit → stops at null/undefined
- Interview tips: explain spread vs rest; Arrow has no own this
- Examples: clone obj → { ...obj }; safe access → obj?.x ?? def



Event Delegation & Bubbling

★ Overview: Event bubbling goes child to parent - delegation uses this to handle many children via single parent listener with e.target.

Step-by-step:
Event Delegation
& Bubbling



★ Why use Event Delegation & Bubbling?

- Save memory - 1 listener vs 100s
- Works for dynamic elements
- Cleaner code
- Leverages bubbling phase

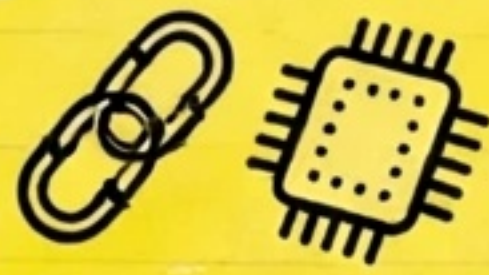
✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ Many child elements
- ✓ Dynamic elements added later
- ✓ Single listener on parent
- ✓ List or table interactions
- ✓ Keywords: bubbling • capturing • delegation • target • currentTarget



COMPLEXITY & DATA:

- Time: $O(1)$ listeners vs $O(n)$
- Space: $O(1)$ - saves memory
- Pitfall: e.target vs e.currentTarget confusion
- Use stopPropagation carefully
- Common examples: Todo list delete, Table row click, Dropdown menu
- Interview Q: Explain bubbling vs capturing



Shadowing + Memoization

★ Overview: Shadowing → inner variable shadows outer • var shadows outer, illegal shadowing with let/const in same scope. Memoization → caches fn results with Map to avoid recompute for speed.

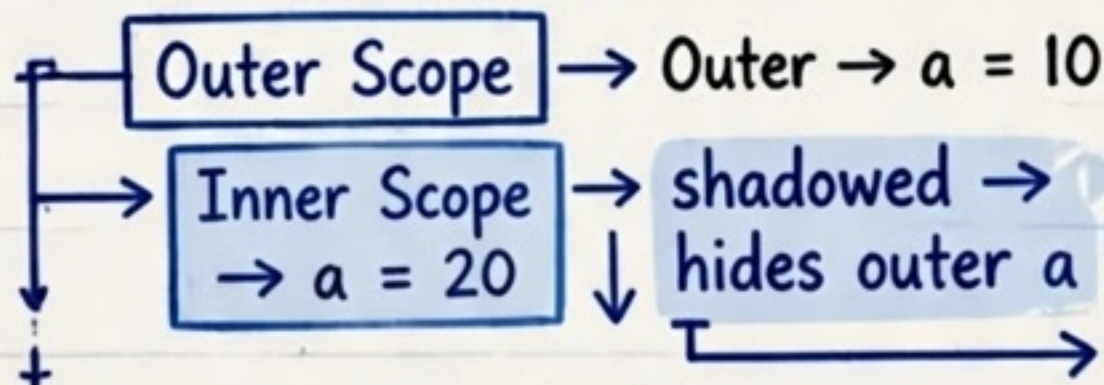
Step-by-step visual:

LEFT — SHADOWING

let a = 10; // outer

function inner() {
 let a = 20; // shadows outer
}

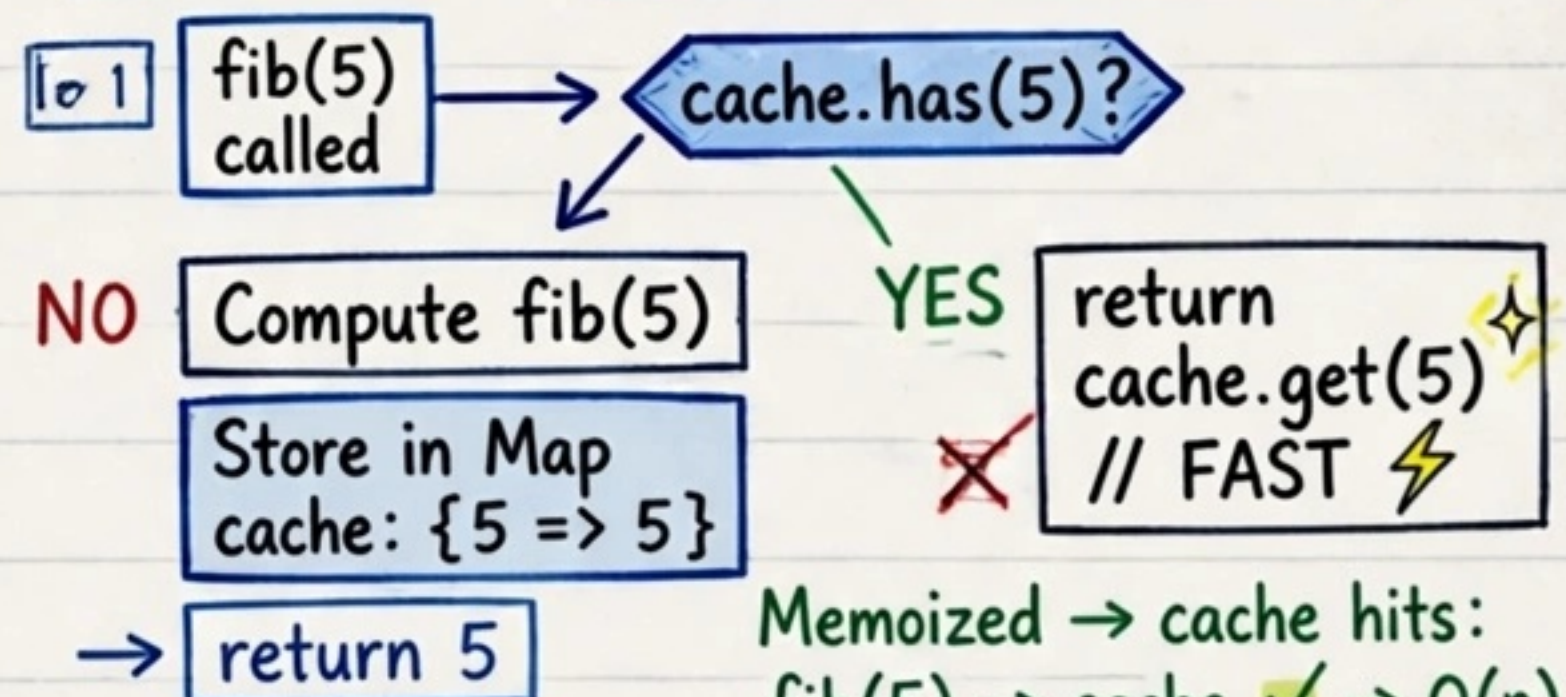
→ Inner a = 20 shadows outer a = 10



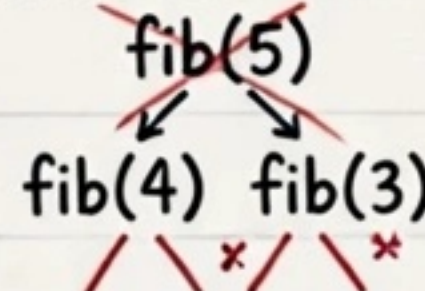
var allows • let/const →
Illegal in same scope ✗

// var shadows ✓
let/const same scope ✗

RIGHT — MEMOIZATION



✗ Naive recursion tree:



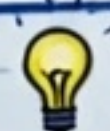
```
const cache = new Map();
function memoFib(n);
if (cache.has(n)) return cache.get(n);
const res = fib(n-1) + fib(n-2);
return res;
```

★ WHY USE Shadowing + Memoization?

- Shadowing: Isolate vars in blocks/functions
- Memoization: $O(2^n) \rightarrow O(n)$ for Fibonacci
- Avoid recompute, speed up expensive fns
- Core for React memo/useMemo optimization

✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ Same name in inner & outer scope
- ✓ var shadows outer, let/const illegal same scope
- ✓ Cache for expensive fn calls
- ✓ Fibonacci, factorial, API cache
- ✓ React.memo / useMemo
- ✓ Keywords: shadow • cache • memo • Map • closure



COMPLEXITY & DATA:

- Time: Memo Fibonacci $O(n)$ vs $O(2^n)$
- Space: $O(n)$ for Map cache
- Pitfalls: Cache bloat, stale cache, memory leak
- Tips: Use Map, limit cache size, pure fn only
- Interview: Explain illegal shadowing gap + memo fib impl
- Ex: memoFibonacci(n), React memo

Final JS Interview Cheat Sheet



★ Overview: Quick revision of all 13 JS concepts — one liner, use case, common interview trap for last minute prep.



#	Concept (with icon)	One-Liner	When to Use	Interview Trap
1	🔗 Closure	fn inside fn scope <code>let count=0; (3) count++</code>	Data privacy, memoization	Loop var bug
2	↑ Hoisting	declarations moved to top <code>var x;</code>	var vs let/const	TDZ
3	🔄 Event Loop	stack empties then queue FIFO <code>queueMicrotask()</code>	async tasks	micro vs macro
4	🕒 Promises & Async/Await	async handling <code>Promise.then</code>	API calls	forget await
5	🔗 Prototypes	object fallback to <code>__proto__</code>	inheritance	hasOwnProperty
6	🎯 This keyword	context depends on call site <code>.call(this)</code>	object methods	arrow has no this
7	🌳 Scope Chain	<code>let x=1; { let x=2 }</code>	shadowing	func borrowing
8	↔ Currying	$f(a,b) \rightarrow f(a)(b)$	reusable partial fn	over-currying function
9	🕒 Debounce & Throttle	control fn call <code>debounce(fn, 300)</code> <code>const {a,b} = obj</code>	modern code	transpilation
11	⚡ ES6+ Features Delegation	parent listens for child <code>parent.addEventListener</code>	dynamic lists	stopPropagation
13	🧠 Memoization / Shadowing	cache + inner var hides outer <code>const cache = {}; 1;</code>	optimization, scope trap	cache invalidation



WHY USE Final Cheat Sheet?

- O(1) revision — all in one page
- Quick recall before interview
- Covers closure to event loop traps
- Boost confidence last minute



✓ HOW TO RECOGNIZE THE PATTERN:

- ✓ Interview revision
- ✓ Quick recall
- ✓ All concepts at glance
- ✓ Last minute prep
- ✓ Keywords: cheat sheet • revision • interview



COMPLEXITY & DATA:

- Time: O(1) glance revision
- Space: Brain O(n) concepts
- Common pitfalls: this, closure, hoisting
- Interview tips: say use case + example
- Examples: debounce search, closure private counter